ables and $C \times 7 \times 61 \times 71$ for the upper-air variables. After patch embedding, these two outputs are concatenated along the channel dimension, creating a combined input with a shape of $C \times 8 \times 61 \times 71$.

The input data, with a size of $C \times 8 \times 61 \times 71$, is processed through a 3D Swin-Transformer (Z. Liu et al., 2021) consisting of three layers. The first layer has 3 blocks and is followed by a patch merging operation that downsamples the data to $2C \times 8 \times 31 \times 36$. In the second layer, which contains 9 blocks, the data is further processed and then upsampled back to its original size of $C \times 8 \times 61 \times 71$ before being passed into the third layer, which also has 3 blocks. The output of the first layer is added to the output of the second layer through a skip connection. The outputs from both the first and second layers are added to the third layer's output via skip connections.

For generating the final output, we validated two methods. In the first method (Upsampler1), we used a 3D convolution with a kernel size of $(8 \times 1 \times 1)$, followed by bilinear upsampling, which doubled the height and width dimensions as a shape of C $\times 1 \times 121 \times 141$. We then applied a 2D convolution with bilinear upsampling to restore the data to its original resolution of $1 \times 241 \times 281$. In the second method (Upsampler2), we used a 3D convolution with a $(8 \times 1 \times 1)$ kernel and a GELU activation to reduce the height dimension to 1. Afterward, we adopted an image reconstruction technique used by SwinIR (Liang et al., 2021), which includes several convolutional layers: one before upsampling, followed by others for further processing. These layers consist of LeakyReLU activations and multiple convolutions to refine the output, ultimately restoring the high-resolution data. A comparison of the results produced by these two methods can be found in the 4.1.

3.2.3 Weighted loss function

The loss function used is the weighted L2 and SIMM (Structural Similarity Index Measure) loss, which is defined as follows:

$$Loss = \lambda 1 * (\hat{X}_{i,j} - X_{i,j})^2 + \lambda 2 * (1 - SSIM(\hat{X}_{i,j} - X_{i,j})) \tag{5}$$

where $\lambda 1$ and $\lambda 2$ are weights that balance L1 and SSIM, is set to 0.5 and 1.5 respectively. The SSIM loss (Wang et al., 2004) is sensitive to changes in contrast, luminance, and texture, making it more effective at preserving the spatial distribution and intensity of features like heavy rainfall.

3.3 Probabilistic model

The probabilistic model is designed to generate residual precipitation, using the current atmospheric state plus mean precipitation as a conditional input. Two key approaches are employed in this model: Variational Autoencoder (VAE) and Conditional Diffusion.

3.3.1 Variational autoencoder

To improve the training efficiency of the diffusion model, we use a VAE model to encode the residual precipitation data, with dimensions $1 \times 900 \times 1400$, into a latent representation of size $16 \times 90 \times 140$. The VAE is used to encode samples from the pixel space to a continuous latent space and then decode them back to the pixel space (Kingma & Welling, 2022). We train our autoencoder models following Rombach et al. (2022), which uses an adversarial manner to enhance the generative quality of the model. The encoder and decoder components of the VAE in this study are constructed as 2D convolutional networks. In the encoder, the first three layers contain two ResNet-type residual blocks and a downsampling block, each layer reduces the spatial dimensions by a factor of 2, while the final layer consists of two ResNet blocks without downsampling. Since our tar-

get size is not a power of 2, the feature maps are interpolated to the target resolution 90×140. The decoder mirrors the encoder structure, using upsampling to progressively restore the spatial resolution, and interpolation to match the final output size to the input. The latent space is regularized using a Kullback–Leibler (KL) divergence loss to align the latent variables with a multivariate standard normal distribution.

3.3.2 Conditional latent diffusion model

The conditional diffusion model we use is based on the Denoising Diffusion Probabilistic Model (DDPM), which progressively corrupts a data sample by adding Gaussian noise over a series of timesteps. A neural network is then trained to reverse this noising process, recovering the original data. The conditional latent diffusion model train-

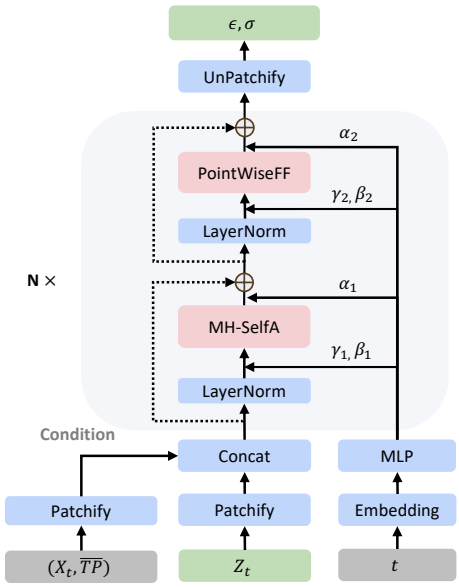

Figure 3. Conditional diffusion model architecture (Peebles & Xie, 2023). We introduce conditioning inputs by using the atmospheric state and mean precipitation.

ing objective can be formulated as:

$$\mathcal{L} = \mathbb{E}_{embedding(x),y,\epsilon\sim\mathcal{N}(0,1),t} \left\|\epsilon - \epsilon_\theta(Z_t, t, cond)\right\|^2 \tag{6}$$

where x is the condition, which is the atmospheric state, y is the residual precipitation, ϵ is random noise, $t \in [1, 1000]$ is the time step of the denoising process, z_t is the noisy latent-space sample at step t, ϵ_θ is the denoiser and θ represents the trainable parameters of the networks. In our setup, we train the denoise model conditioned on ERA5 surface variables (T2m, U10m, V10m, MSLP and TP) and upper air variables at 13 pressure levels (T, U, V, SP, and Z) to guide the denoising process and improve the accuracy of the generated residual precipitation. During inference, we replace ERA5 precipitation with the predicted 1-hour precipitation from the deterministic model.

The denoising network ϵ_θ of our model referred to the DiT (Diffusion Transformer) architecture, a transformer-based model specifically designed for diffusion processes (Peebles & Xie, 2023). DiT utilizes self-attention mechanisms to capture both global and local dependencies in the data. The conditioning variables are first processed through a 2D patch embedding as the improved approach we proposed in 3.2.2. The patch size for the

conditional variables is 4×4, while the noise state has a patch size of 2×2. After patchifying, both the conditional input and the noise state have the same spatial dimensions of 45×75. The patch-embedded conditional input is then concatenated along the channel dimension with the noisy state of the diffusion model at each timestep.

Since CMPA coverage region is smaller than the area used for training the deterministic model with ERA5, the ERA5 variables are cropped to the region of 15°N-60°N and 70°E-140°E (181 × 241) before being input into probabilistic model.

3.4 Model training and inference

The deterministic model is trained on ERA5 data from 2001 to 2019. For the probabilistic model, the training dataset spans the years 2018-2019, with 2020 used as the validation set and 2021 reserved for testing. The deterministic model was trained for approximately 69 hours on 2 Nvidia A100 GPUs, while the probabilistic model took 38 hours to train.

During the inference phase, the probabilistic model first samples from the latent space and then uses ERA5 variables and precipitation predictions from the deterministic model as conditioning inputs. Sampling is performed using the Denoising Diffusion Implicit Models (DDIM) approach (Song et al., 2022), with 300 sampling steps. This process is repeated for 11 iterations, generating 11 ensemble forecast members, which provides a probabilistic estimate of precipitation.

4 Results

4.1 Ablation study of deterministic model

Due to the tendency of deterministic models to underestimate precipitation intensity and blurriness, we adopted several methods to improve the performance of precipitation forecasting. These methods are as follows (For more details about these methods, refer to 3.2.2:

1. Exp-d1: Weighted MSE and SSIM loss: MSE ensures pixel-wise accuracy, while SSIM helps the model preserve the spatial distribution and intensity of features like heavy rainfall.
2. Exp-d2:Incorporation of static and temporal (ST) features: Enables the model better learn local and temporal variations.
3. Exp-d3:Non-linear patch embedding: Enhances the patch embedding by applying depth-wise convolution and activation layers, allowing each patch to extract more non-linear interactions.
4. Exp-d4:Refined upsampler: Employs additional convolutional layers in the upsampling decoder to improve the recovery of precipitation details.

The data from 2016 to 2019 were used as training dataset for the ablation study, with August, 2020 serving as the validation set. Figure 4 shows the CSI scores of different experiments under various rainfall thresholds. From Figure 4, it is evident that all methods improved CSI scores for precipitation, with the enhancement becoming more pronounced as the precipitation intensity increases. In exp-d1, the use of weighted SSIM and MSE loss, compared to using only MSE as the loss function, improved the CSI scores for rainfall above thresholds of 2 mm, 5 mm, 10 mm, 15 mm, and 20 mm by 3.3%, 7.3%, 16.3%, 33%, and 50%, respectively. Building on exp-d1, we introduced static and temporal features in exp-d2. When compared to exp-d1, exp-d2 shows improvements at all thresholds, specifically by 1.3%, 3.2%, 5.1%, 14.6%, 34%, and 56.2%. Exp-d3 introduces nonlinear patch embedding compared to exp-d2. While the improvements are not very significant for lighter rainfall, with no noticeable gain at the 0.1 mm/h threshold and only